



Programación de Aplicaciones Gráficas

Sesión 2: Organizando el proceso de rendering y la interfaz con clases especializadas

Introducción.....	1
 ¿Qué es un patrón de diseño?.....	2
 ¿Qué es el desacoplamiento entre módulos?.....	2
 Bibliotecas de interfaz.....	2
Objetivos.....	3
Material.....	3
Proceso.....	3
 1. Añadir una clase <code>Renderer</code> a tu proyecto, dentro del espacio de nombres <code>PAG</code>.....	3
 2. Escribir el contenido de <code>PAG::Renderer</code> siguiendo el patrón <code>Singleton</code>.....	3
 3. Llamar a los métodos de <code>PAG::Renderer</code> desde el archivo <code>main.cpp</code>.....	7
 4. Añadir <code>Dear ImGui</code> al proyecto.....	8
 5. Patrón observador.....	12
Trabajo autónomo.....	14
 Entrega.....	15

Introducción

Tal y como vimos en la sesión anterior, GLFW proporciona una serie de funciones que permiten a nuestras aplicaciones gráficas, basadas en OpenGL, generar ventanas capaces de mostrar un área de rendering con un contexto gráfico **Core Profile** asociado, así como de recibir eventos de usuario (teclado, ratón, etc.).

Aunque en muchos ejemplos de libros y sitios web toda la gestión del proceso de *rendering* se lleva a cabo dentro de la función `main()` de los programas, esa no es la forma correcta de organizarlo. En estas prácticas vamos a aprender cómo estructurar nuestra aplicación gráfica de la forma más desacoplada posible. Para ello haremos uso de patrones de diseño.

¿Qué es un patrón de diseño?

A lo largo de los años, muchos programadores han tenido que enfrentarse a problemas parecidos de distinta índole: relacionados con las interfaces de las aplicaciones, con la gestión de memoria, con la creación de objetos, etcétera. Poco a poco, la comunidad de programadores ha ido convergiendo en una serie de soluciones habituales para problemas similares, y estas soluciones han terminado convirtiéndose en estándares de diseño, a los que se le ha dado el nombre de **patrones**.

Un **patrón de diseño**, por tanto, es una solución bien conocida y probada a un problema de diseño de software concreto. Hay muchos recursos sobre patrones de diseño, tanto en la [web](#) como en libros en papel u online. A lo largo de estas prácticas aplicaremos algunos de ellos, y los iremos explicando.

¿Qué es el desacoplamiento entre módulos?

Una buena práctica de programación es escribir código lo más desacoplado posible. Existe **desacoplamiento entre módulos** (clases, bibliotecas, funciones...) cuando los cambios en un módulo no provocan efectos colaterales no deseados en los otros módulos. Si nos referimos a clases, se pueden hacer cambios en una clase sin que haya que hacer ningún cambio en las clases que la utilizan. Para que exista desacoplamiento hay que limitar el conocimiento que unas clases tienen de las otras. Hay que tener en cuenta que la clase A **conoce** a la clase B cuando:

- A tiene algún atributo de clase B.
- Algún método de A toma como parámetro un objeto de clase B.
- Algún método de A devuelve algún valor de clase B.

La mejor manera de conseguir desacoplamiento es asignando correctamente las responsabilidades de todas las clases. Como norma general, **cada clase debería dedicarse a una sola tarea**. Si en una clase se mezclan diferentes responsabilidades, probablemente el diseño de clases no sea correcto y haya que replantearlo.

Bibliotecas de interfaz

En la actualidad, hay muchos proyectos en marcha de bibliotecas para facilitar el desarrollo de aplicaciones con interfaces de usuario más o menos sofisticadas, incluyendo controles de distintos tipos como botones, deslizadores, selectores de color, etcétera. La mayoría aplican unos principios de diseño similares, por lo que una vez que dominamos una biblioteca, adaptarnos a otra diferente no debería ser muy complicado.

En estas prácticas vamos a utilizar [Dear ImGui](#), una biblioteca sencilla que nos va a permitir añadir controles típicos a nuestra aplicación. Veremos cómo conectarla con una aplicación basada en OpenGL y GLFW, intentando hacer un código lo más desacoplado posible.

Objetivos

En esta sesión aprenderemos cómo organizar el proceso de *rendering* y la respuesta a los eventos con una clase especializada a la que llamaremos `Renderer`, dentro de un espacio de nombres PAG. También aprenderemos a construir esa clase de forma que pueda ser utilizada desde cualquier módulo de código, a la vez que garantizamos que sólo existe una instancia de la misma. Para ello, primero aprenderemos qué es el **patrón Singleton**, cómo se construye y cómo se usa.

También aprenderemos a incorporar Dear ImGui a nuestro proyecto, y crearemos un par de controles básicos.

Material

Para esta segunda sesión necesitarás el proyecto con GLFW y GLAD que preparaste en la práctica anterior.

Como cada práctica se construye sobre la anterior, es **muy recomendable** que crees un **duplicado del proyecto**, o bien utilices algún **sistema de control de versiones**. Así podrás tener siempre guardada una copia de cómo estaba todo antes de cada práctica, por si necesitas volver atrás para revisar tu código.

Proceso

Los pasos a seguir en esta práctica son:

1. Añadir una clase `Renderer` a tu proyecto, dentro del espacio de nombres PAG

Añade una clase nueva a tu proyecto, utilizando para ello un módulo aparte (es decir, un archivo `.h` y un archivo `.cpp` nuevos). Asegúrate de crear la clase dentro de un [espacio de nombres](#) denominado PAG.

2. Escribir el contenido de `PAG::Renderer` siguiendo el patrón Singleton

El Singleton es un patrón de diseño pensado para resolver el siguiente problema: tenemos una clase de la que **sólo puede haber una instancia** (un objeto) durante toda la ejecución de la aplicación. Además, este objeto debe estar **accesible desde cualquier módulo** de la aplicación (Figura 1).

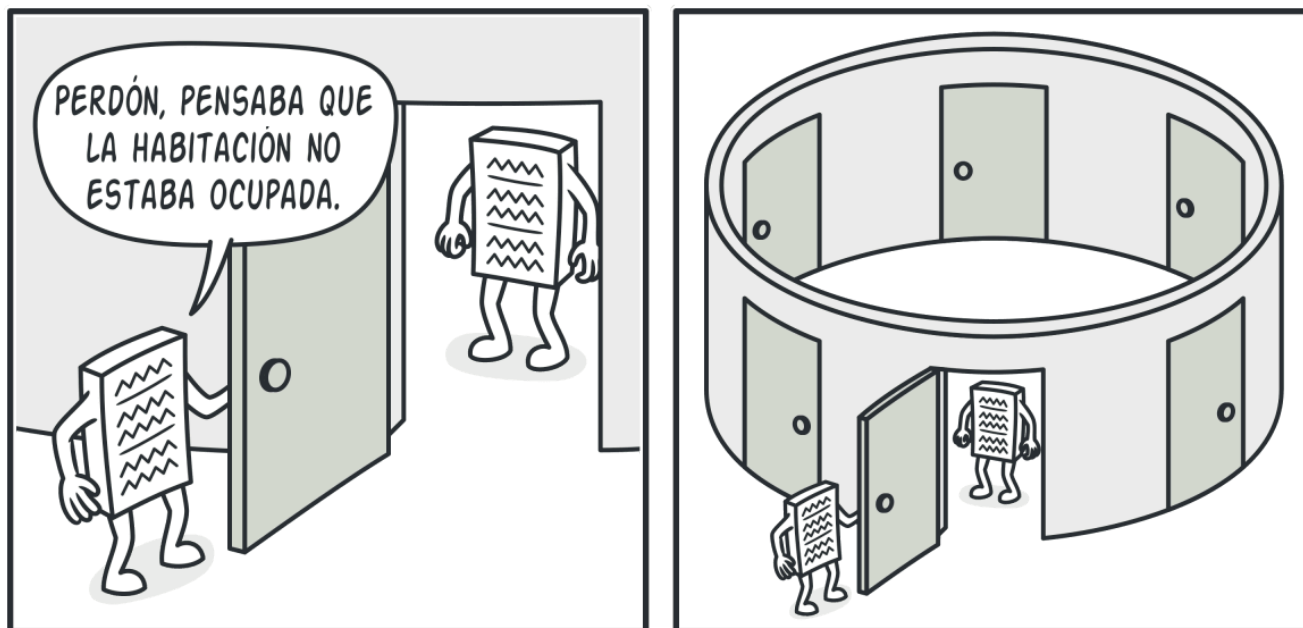


Figura 1. Patrón Singleton. Siempre se obtiene la misma instancia de la clase diseñada utilizando el patrón Singleton, independientemente del lugar desde el que se haga la llamada para obtener la instancia única. Fuente: [Refactoring Guru](https://refactoring.guru/).

La forma de resolver este problema aplicando este patrón es a través de un **atributo de clase con visibilidad privada** y de tipo puntero a la misma clase. Además, se le da al **constructor visibilidad privada**, para evitar que se puedan crear objetos desde otros módulos. Este atributo suele recibir el nombre de **instancia**. Aplicando esto a la práctica, tendríamos esto en el archivo `Renderer.h`:

```
// En Renderer.h
private:
    static Renderer* instancia;
    // ...
    Renderer ();
public:
    // ...
```

Al tener visibilidad privada, el atributo sólo será accesible a través de métodos de la clase, y al estar asociado a la clase y no a cada objeto, sólo existirá una ocurrencia de este atributo en la aplicación.

Nos falta ver cómo gestionar el objeto asociado a este atributo. La forma de hacerlo según el patrón **Singleton** es asignándole valor inicial **nullptr** y haciendo uso de “**inicialización perezosa**” (*lazy initialization*); esto es: crear el objeto la primera vez que sea necesario, y no al iniciar la aplicación.

Recuerda que en C++, la forma de inicializar los atributos de clase es fuera de la declaración de la clase y de la implementación de los métodos, y preferentemente al principio del archivo `.cpp`:

```
// En Renderer.cpp
PAG::Renderer* PAG::Renderer::instancia = nullptr;
```

Además, necesitamos un método para acceder a la instancia, ya que es un atributo de clase con visibilidad privada. Crearemos este método como método de clase:

```
// En Renderer.h
static Renderer& getInstance ();
```

```
// En Renderer.cpp
PAG::Renderer& PAG::Renderer::getInstance ()
{ if ( !instancia ) // Lazy initialization: si aún no existe, lo crea
  { instancia = new Renderer ();
  }
  return *instancia;
}
```

¡OJO! Puedes tener la tentación de utilizar una variable de tipo puntero para guardar la dirección de memoria que te devuelve el método `getInstance`. Esto NO ES UNA BUENA PRÁCTICA, sino que debes llamar siempre a `getInstance` para acceder a los métodos de la clase.

El uso de inicialización perezosa en las aplicaciones gráficas es importante, porque en muchas ocasiones tendremos objetos que precisarán que, por ejemplo, ya se haya creado el contexto OpenGL o que la ventana esté visible antes de crearlos.

Implementaremos el resto de los métodos de la clase `Renderer` como métodos normales. Por el momento, la clase `Renderer` es bastante sencilla, y sólo incorporará los métodos para responder a los eventos desde el `main.cpp`.

Aquí tienes un ejemplo de cómo podría ser el archivo `Renderer.h`:

```
...

/**
 * @file Renderer.h
 * @author algarcia
 *
 * @date 2021/08/06
 *
 * @brief Declaración de la clase Renderer
 */

/**
 * Espacio de nombres para las prácticas de Programación de Aplicaciones
 * Gráficas
 */
namespace PAG
{ /**
 * @brief Clase encargada de encapsular la gestión del área de dibujo
 * OpenGL
 *
 * Esta clase coordina el renderizado de las escenas OpenGL. Se implementa
 * aplicando el patrón de diseño Singleton. Está pensada para que las
```

```

    * funciones callback hagan llamadas a sus métodos
    */
class Renderer
{ private:
    static Renderer* instancia; ///< Puntero al único objeto

    Renderer ();

public:
    virtual ~Renderer ();
    static Renderer& getInstancia ();
    void refrescar ();
};
}

```

Y aquí tienes un ejemplo de cómo podría ser el archivo `Renderer.cpp`:

```

/**
 * @file Renderer.cpp
 * @author algarcia
 *
 * @date 2021/08/06
 *
 * @brief Implementación de la clase Renderer
 */

#include <GL/gl.h>

#include "Renderer.h"

namespace PAG
{
    Renderer* Renderer::instancia = nullptr;

    /**
     * Constructor por defecto
     */
    Renderer::Renderer ()
    { }

    /**
     * Destructor
     */
    Renderer::~~Renderer ()
    { }

    /**
     * Consulta del objeto único de la clase
     * @return La dirección de memoria del objeto
     */
    Renderer& Renderer::getInstancia ()
    { if (!instancia)
        { instancia = new Renderer;
        }
    }
}

```

```

    return *instancia;
}

/**
 * Método para hacer el refresco de la escena
 */
void Renderer::refrescar ()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
}
}

```

Una vez implementada la clase `Renderer`, nos queda conectarla con el resto del proyecto.

3. Llamar a los métodos de `PAG::Renderer` desde el archivo `main.cpp`

Deberemos incluir en la función `main` y en las funciones de *callback* aquellas llamadas a los métodos de `PAG::Renderer` que sean necesarias.

Para lograr un código bien desacoplado, debes hacer que **las llamadas a GLFW** (que a fin de cuentas, son relativas fundamentalmente a la interfaz de la aplicación) **se queden en el archivo `main.cpp`**, mientras que **en `PAG::Renderer` tendremos las llamadas a las funciones de OpenGL** (Figura 2). Esto nos permitirá que, si cambiamos la forma de gestionar la interfaz de nuestra aplicación gráfica (utilizando otra biblioteca en lugar de GLFW), no tengamos que cambiar nada en nuestra clase `PAG::Renderer`.

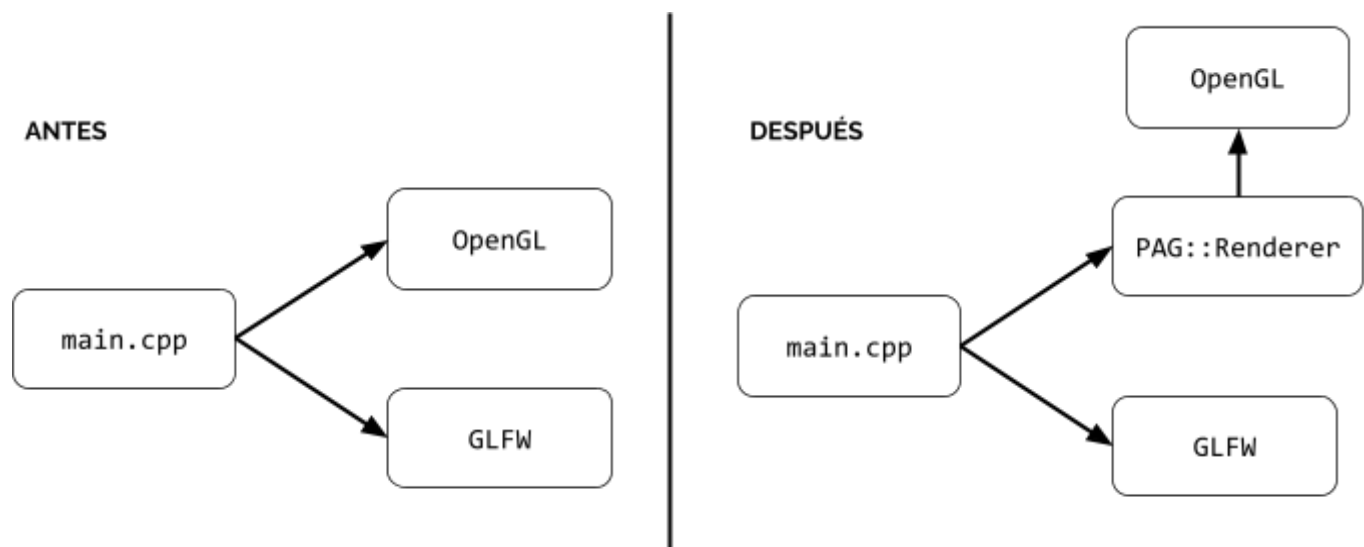


Figura 2. A la izquierda se representa la situación actual, en la que desde el `main.cpp` se llama a funciones de OpenGL y GLFW. A la derecha se muestra cómo debe quedar tu aplicación.

Por ejemplo, el *callback* para refrescar la ventana del archivo `main.cpp` quedaría de la siguiente forma:

```
#include "Renderer.h"
// ...
void callbackRefrescoVentana ( GLFWwindow* ventana )
{
    PAG::Renderer::getInstancia().refrescar();
    glfwSwapBuffers (ventana);
    std::cout << "Finaliza el callback de refresco" << std::endl;
}
```

Puede ser necesario añadir nuevos atributos a la clase `PAG::Renderer`, como el color de fondo, así como nuevos métodos, como uno para inicializar OpenGL con las opciones que se van a mantener siempre igual durante toda la ejecución de la aplicación, o uno para consultar las capacidades de OpenGL disponibles y luego mostrarlas por consola (obviamente, se mostrarán desde el `main.cpp`).

4. Añadir Dear ImGui al proyecto

Una vez añadida la clase `PAG::Renderer`, vamos a ver cómo añadir Dear ImGui a nuestra aplicación. Como es una biblioteca relativamente simple, nos va a dar menos problemas si descargamos el código fuente (de manera similar a como se podía hacer con GLAD) desde [su repositorio de GitHub](#).

Solo es necesario descargar:

- Los archivos C++ de la carpeta raíz del proyecto:
 - `imconfig.h`
 - `imgui.cpp`
 - `imgui.h`
 - `imgui_demo.cpp`
 - `imgui_draw.cpp`
 - `imgui_internal.h`
 - `imgui_tables.cpp`
 - `imgui_widgets.cpp`
 - `imstb_rectpack.h`
 - `imstb_textedit.h`
 - `imstb_truetype.h`
- Los archivos C++ para conectar Dear ImGui con OpenGL y GLFW. Estos archivos son:
 - `backends/imgui_impl_glfw.cpp`
 - `backends/imgui_impl_glfw.h`
 - `backends/imgui_impl_opengl3.cpp`
 - `backends/imgui_impl_opengl3.h`
 - `backends/imgui_impl_opengl3_loader.h`

Lo más cómodo será crear una carpeta en tu proyecto (por ejemplo, puedes llamarla “`imgui`”), y copiar dentro de ella todos estos archivos. Para organizar mejor los archivos, puedes crear dos subcarpetas: una llamada “`include`” (por ejemplo) que contenga los archivos `.h`, y otra llamada “`src`” (por ejemplo) que contenga los archivos `.cpp`. Esa es la forma en que lo vamos a considerar en el resto de las prácticas.

Recuerda que habrá que editar el archivo `CMakeLists.txt` para que se compilen todos los fuentes de Dear ImGui en tu proyecto. Una posible forma de hacerlo es la siguiente (fíjate únicamente en cómo agrupamos los nombres de archivo y cómo los pasamos a compilación; el uso de las bibliotecas no cambia de como tú lo tengas en tu proyecto):

```
cmake_minimum_required(VERSION 3.27)
project(pr02)

set(CMAKE_CXX_STANDARD 17)

# Define variables agrupando nombres de archivo
file( GLOB MY_FILES *.cpp )
file( GLOB GLAD_FILES GLAD/src/*.c )
file( GLOB IMGUI_FILES imgui/src/*.cpp )

add_executable(${PROJECT_NAME} ${MY_FILES}
                                   ${GLAD_FILES}
                                   ${IMGUI_FILES})

target_include_directories( ${PROJECT_NAME} PUBLIC GLAD/include
                                                                    imgui/include )
target_link_libraries( ${PROJECT_NAME} GL glfw )
```

Como en nuestro proyecto utilizamos GLFW para interactuar con el gestor de ventanas del sistema y OpenGL para dibujar, tendremos que añadir en nuestro código los includes correspondientes:

```
#include <imgui.h>
#include <imgui_impl_glfw.h>
#include <imgui_impl_opengl3.h>
```

Para la inicialización de Dear ImGui, tenemos que llamar a las siguientes funciones una sola vez:

```
ImGui_CHECKVERSION();
ImGui::CreateContext ();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
```

Y para el caso de GLFW y OpenGL, hay que completar la inicialización con las siguientes llamadas:

```
// Aquí w es el puntero a la ventana GLFW (GLFWwindow*)
ImGui_ImplGlfw_InitForOpenGL ( w, true );
ImGui_ImplOpenGL3_Init ();
```

Una vez que Dear ImGui está inicializada en nuestra aplicación, debemos redibujar toda la interfaz cada vez que se refresque la imagen. Esto es así porque Dear ImGui se dibuja encima del contenido del viewport OpenGL. Para hacer esto, tenemos que incluir las siguientes llamadas en nuestro código (por ejemplo, en el ciclo de eventos de la aplicación):

```
ImGui_ImplOpenGL3_NewFrame();
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();
// Se dibujan los controles de Dear ImGui
```

```
// Aquí va el dibujado de la escena con instrucciones OpenGL
ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData ( ImGui::GetDrawData() );
```

Los controles en Dear ImGui están organizados por defecto en ventanas flotantes encima del contenido de la ventana. Estas ventanas se pueden contraer y expandir, así como redimensionar (Figura 3). También es posible trabajar con ventanas empotradas en la ventana principal de la interfaz, pero con el uso básico será suficiente en nuestro caso.

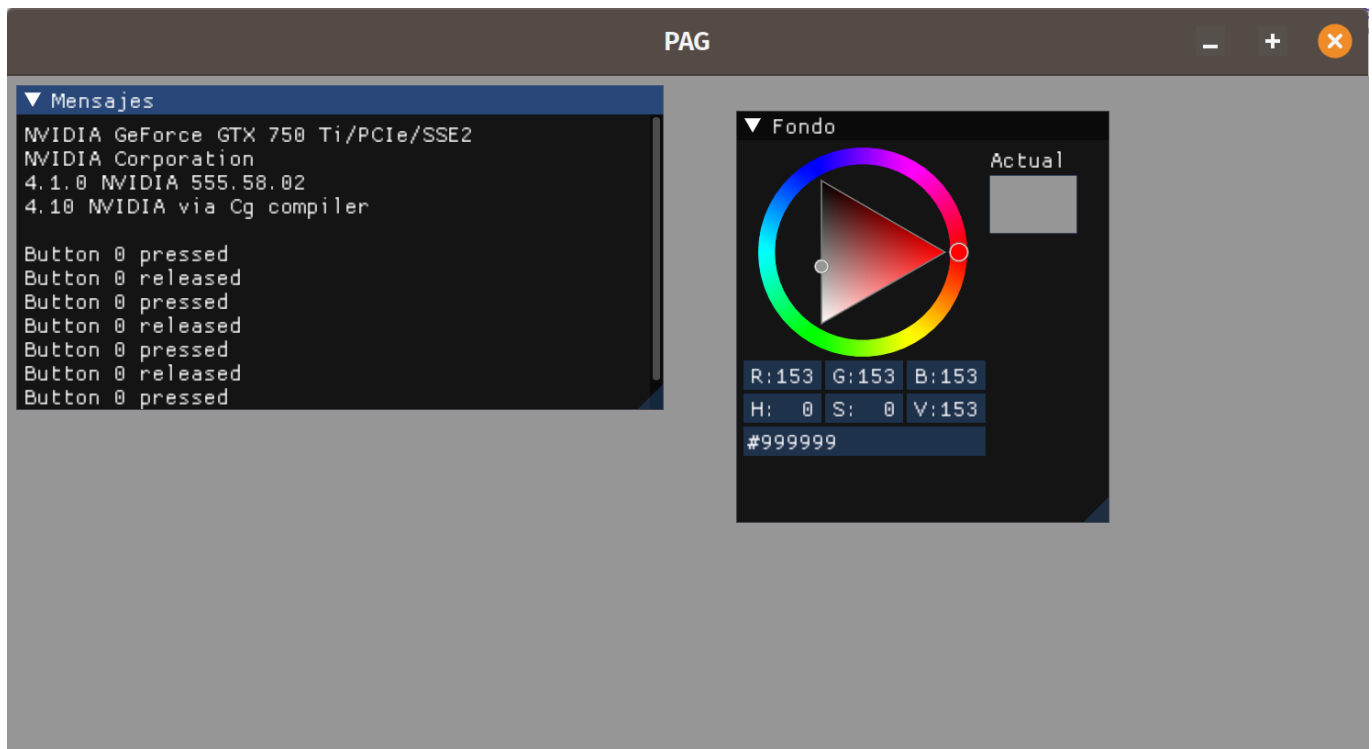


Figura 3. Ventanas flotantes de Dear ImGui.

Para crear una ventana flotante de Dear ImGui hay que indicar en primer lugar en qué coordenadas de la ventana principal está situada. Esto lo hacemos de la siguiente manera:

```
ImGui::SetNextWindowPos ( ImVec2 (10, 10), ImGuiCond_Once );
```

Con esto, estamos indicando en qué posición de la ventana (X, Y) queremos que aparezca. El flag que hemos puesto a continuación sirve para asegurar que cada vez que se ejecuta la aplicación, la ventana aparece en el mismo sitio. Si no lo incluyéramos, las ventanas aparecerían en la última posición que tuvieron en la ejecución anterior (el archivo `imgui.ini` que se genera automáticamente en el directorio de trabajo, guarda la posición de las ventanas antes de finalizar la aplicación). Llamando a la función `ImGui::Begin`, y pasándole como parámetro el título de la ventana, la creamos (en el ejemplo de código de más abajo lo puedes ver).

Cada una de las ventanas puede contener uno o varios controles. A la hora de actualizar el contenido de la interfaz, es necesario comprobar primero si la ventana flotante está contraída o desplegada, porque obviamente, si está contraída, no es necesario dibujar los controles. Esto se puede hacer con un código similar a este:

```

if ( ImGui::Begin ( "Mensajes" ) )
{ // La ventana está desplegada
  ImGui::SetWindowFontScale ( 1.0f ); // Escalamos el texto si fuera necesario
  // Pintamos los controles
}

// Si la ventana no está desplegada, Begin devuelve false
ImGui::End ();

```

En el código anterior, es importante la llamada a `SetWindowFontScale` cuando trabajamos con monitores de alta resolución. Si no escalamos el texto de los controles, estos se verán muy pequeños, y la interfaz no será fácil de utilizar. La llamada a esta función es independiente de una ventana de Dear ImGui a otra, así que si tenemos más de una ventana en la interfaz, deberemos repetirla para cada una de ellas.

Cada control de Dear ImGui tiene una forma diferente de pintarlo, dependiendo de aspectos como el tipo de control, si ese control modifica alguna variable de nuestro código, etcétera. Puedes ver todos los posibles controles de Dear ImGui y ejemplos de su uso a través del [manual interactivo de @pthom](#), que te permite navegar por los controles y ver el fragmento de código que se utiliza para insertarlo. También puedes revisar el archivo `imgui_demo.cpp` para ver cómo se usan los controles.

Dear ImGui tiene sus propios callbacks para eventos como la pulsación de los botones del ratón. Para que no interfiera con GLFW, debemos comunicar desde nuestros callbacks GLFW los eventos acaecidos a Dear ImGui con un código similar a este (este ejemplo es para botones del ratón):

```

void mouseButtonCallback ( GLFWwindow* window, int button, int action, int mods )
{ if ( action == GLFW_PRESS )
  { // Si es necesario hacer algo con este evento, indicarlo aquí
    // Finalmente, comunica el evento de ratón a ImGui
    ImGuiIO& io = ImGui::GetIO ();
    io.AddMouseButtonEvent ( button, true );
  }
  else if ( action == GLFW_RELEASE )
  { // Si es necesario hacer algo con este evento, indicarlo aquí
    // Finalmente, comunica el evento de ratón a ImGui
    ImGuiIO& io = ImGui::GetIO ();
    io.AddMouseButtonEvent ( button, false );
  }
}

```

De la misma manera puede que sea necesario con los eventos de teclado, por ejemplo.

Cuando la aplicación finaliza, hay que liberar los recursos ocupados por Dear ImGui antes de destruir la ventana GLFW:

```

ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext ();
glfwDestroyWindow ( window );

```

Para hacer un código lo más desacoplado posible, puedes implementar otra clase aplicando de nuevo el patrón Singleton, de tal forma que toda la comunicación con Dear ImGui se haga a través de esa clase, aislando así el código del `main.cpp` de esta biblioteca (Figura 4). Incluso, puedes desarrollar una jerarquía de clases para encapsular distintos tipos de ventanas y controles, según te interese.

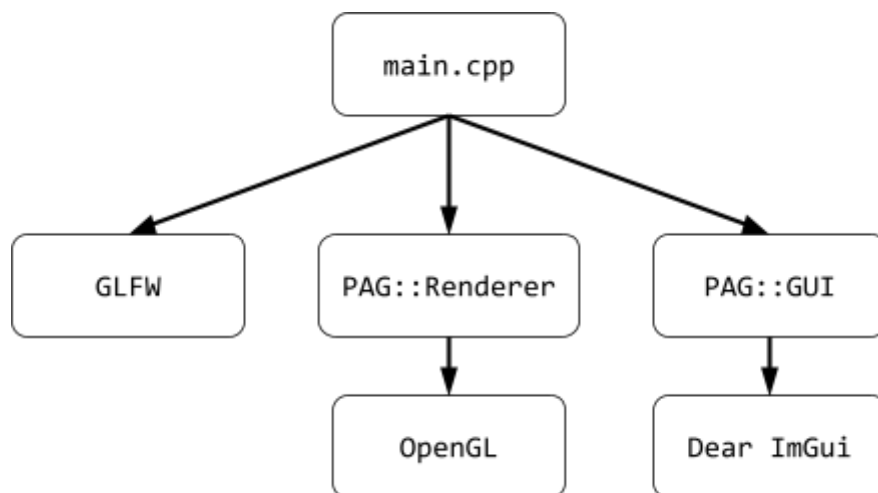


Figura 4. Estructura de la aplicación con clases para desacoplar la interacción con las bibliotecas OpenGL y Dear ImGui.

Hacerlo de esta forma nos facilitará en el futuro cambiar la biblioteca de interfaz.

5. Patrón observador

Por el momento, nuestra interfaz es sencilla, pero conforme vayamos añadiendo controles y opciones, tendremos que asegurar una forma cómoda y eficiente de comunicar las elecciones del usuario al resto de la aplicación. La idea es que nuestra clase `Renderer` sea la que centralice el proceso; sin embargo, vamos a aprovechar esta situación para introducir otro patrón de diseño: [el patrón observador](#).

Este patrón de comportamiento describe cómo organizar el código en aquellas situaciones en que un objeto debe notificar a otros sobre un cambio en sus propiedades (Figura 5).

En nuestro caso, el primer ejemplo es un control de la interfaz que permite cambiar el color de fondo de la ventana. Lo más razonable es que el valor de este color sea un atributo de la propia clase `Renderer`, pero en lugar de consultar a la interfaz por un posible cambio en este color en cada refresco de ventana, es más eficiente que sea la propia interfaz la que notifique a `Renderer` sobre cada cambio.

La forma de resolver esta situación según el patrón observador es la siguiente (estos fragmentos de código son solo ejemplos; adapta el patrón observador a tu diseño y tu código concretos):



Figura 5. En lugar de estar consultando continuamente el estado de un objeto, los observadores se suscriben, de tal forma que es el objeto el que avisa a sus suscriptores de un cambio en su estado.

- Crear una interfaz (recuerda que en C++ no existe un elemento en el lenguaje para las interfaces, pero se puede simular el comportamiento con una clase abstracta) para los observadores. Por ejemplo:

```
class Listener
{
public:
    Listener () = default;
    virtual ~Listener () = default;
    // WindowType es un tipo enumerado propio para identificar
    // el tipo de ventana de la interfaz
    virtual void wakeUp ( WindowType t, ... ) = 0;
};
```

En este código estamos utilizando también otra característica de C++: las [funciones de argumentos variables](#). Esta característica nos permite pasar un número variable de argumentos a un método, e interpretarlos luego como nos interese en el código de dicho método en cada clase que herede de `Listener`.

- En cada clase que gestione una ventana de la interfaz, tendremos un contenedor de punteros a objetos de clase `Listener`, un método para suscribirse (añadirse a esta lista) y otro método para avisar a los observadores del cambio:

```
// ...
std::vector<Listener*> _listeners;

// ...
// MyGUIelement es una clase abstracta que implementa todo el
// comportamiento compartido por las ventanas de la interfaz
void MyGUIelement::addListener ( Listener *listener )
{
    _listeners.push_back ( listener );
}
```

```
// ...
// BgWindow es una clase heredada de MyGUIelement
void BgWindow::warnListeners ()
{ glm::vec3 aux = _datos;

  for ( int i = 0; i < _listeners.size(); i++ )
  { _listeners[i]->wakeUp ( WindowType::Background, &(aux[0])
                           , &_showAxis );
  }
}
```

- En las clases que implementan la interfaz Listener, procesar la llamada al método que notifica el cambio en el estado:

```
void Renderer::wakeUp ( WindowType t, ... )
{ switch ( t )
  { case WindowType::Background:
    { std::va_list args;
      va_start ( args, t );
      _bgColor = *( va_arg ( args, glm::vec3* ) );
      _showAxis = *( va_arg ( args, bool* ) );
      va_end ( args );
      break;
    }
    // Procesar el resto de tipos de ventana
  }
  // Terminar cualquier otro procesamiento que sea necesario
}
```

De esta forma, el control para la selección del color de fondo notifica a la clase `Renderer` únicamente cuando hay un cambio en la selección del usuario.

Como ya hemos comentado, ten en cuenta que los fragmentos de código que mostramos aquí son únicamente ejemplos a título orientativo. El diseño de la aplicación queda a tu elección, y es uno de los aspectos que se valoran en estas prácticas.

Trabajo autónomo

1. **Completa la clase `PAG::Renderer`**, añadiendo los métodos que falten para poder modificar las funciones *callback* que tienes registradas en tu función `main`, de forma que **en lugar de llamar directamente a las funciones de OpenGL, lo hagan a través de `PAG::Renderer`**.
2. Una vez hecho esto, **modifica las funciones *callback*** de tu archivo `main.cpp`, para que llamen a los métodos de `PAG::Renderer` (insistimos: en tu archivo `main.cpp` no puede haber llamadas a funciones de OpenGL, solo llamadas a GLFW, GLAD y `PAG::Renderer`).
3. Añade a tu aplicación la biblioteca Dear ImGui, e incluye en tu aplicación **un control de selección de color** para cambiar el color de fondo de la ventana, así como **una ventana de mensajes**, de tal forma que cualquier mensaje generado desde la aplicación se muestre en esa ventana en lugar de en la consola.

Entrega

1. Sube tu proyecto a un servicio git online (GitHub, GitLab o similar).
2. Añade al `README.md` del proyecto una explicación de los cambios realizados. Si lo ves necesario, incluye un diagrama de clases UML.
3. Configura tu proyecto de tal forma que se pueda acceder de forma anónima al mismo.
4. A través del espacio de la asignatura en la plataforma PLATEA, en la tarea titulada **Práctica 2** que se encuentra en el espacio de prácticas de la asignatura, deberás indicar la URL de acceso a tu proyecto. Si es la misma de la práctica anterior, indícalo.